

ECNet: Robust Malicious Network Traffic Detection With Multi-View Feature and Confidence Mechanism

Xueying Han¹, Song Liu, Junrong Liu, Bo Jiang², Zhigang Lu, and Baoxu Liu

Abstract—Malicious traffic detection in the real world faces the challenge of dealing with a diverse mix of known, unknown, and variant malicious traffic, requiring methods that are accurate, generalizable, and reliable for identifying both known and emerging threats. However, existing methods are unable to fully meet these requirements. Supervised methods can accurately detect known malicious traffic, but their performance declines significantly when encountering unknown attacks. Additionally, the misclassification is usually silent, leading to doubts about the reliability and practicality. Unsupervised methods can deal with unknown attacks, but their high false positive rate and inability to utilize the knowledge of existing attack data constitute obvious shortcomings. To overcome these limitations, we propose ECNet, an end-to-end robust malicious network traffic detection method. Particularly, ECNet incorporates multi-view features, including content and pattern features, and employs a gated-based feature fusion approach, providing an efficient and robust representation. Moreover, ECNet introduces a confidence mechanism and combines category probability and confidence values during training and detection; therefore, it can accurately detect both known and unknown malicious traffic while ensuring the credibility of results. To validate the performance of ECNet, we conduct comprehensive experiments on six reorganized datasets and compare ECNet with seven state-of-the-art methods. The results demonstrate that ECNet outperforms others, particularly showing significant improvements in detecting unknown attacks, with up to a 14.15% increase in F1 compared to the best-performing method.

Index Terms—Malicious network traffic detection, deep learning, feature extraction, confidence mechanism, robustness.

I. INTRODUCTION

WITH the rapid developments of network technology, the challenges faced by cybersecurity are becoming increasingly severe. Attack methods have become more sophisticated and covert, including various advanced malware, phishing attacks, and complex cyber espionage activities [1].

Manuscript received 10 January 2024; revised 30 May 2024; accepted 6 July 2024. Date of publication 10 July 2024; date of current version 18 July 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFC2206402; in part by the Youth Innovation Promotion Association Chinese Academy of Sciences (CAS) under Grant 20211156; in part by the Program of Key Laboratory of Network Assessment Technology, CAS; and in part by the Program of Beijing Key Laboratory of Network Security and Protection Technology. The associate editor coordinating the review of this article and approving it for publication was Dr. Z. Berkay Celik. (Corresponding author: Bo Jiang.)

The authors are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: hanxueying@iie.ac.cn; liusong1106@iie.ac.cn; liujunrong@iie.ac.cn; jiangbo@iie.ac.cn; luzhigang@iie.ac.cn; liubaoxu@iie.ac.cn).

Digital Object Identifier 10.1109/TIFS.2024.3426304

The security of individuals, enterprises, and even nations is all under serious threat. Identifying malicious traffic from a vast amount of network traffic, while reducing false positives and negatives, has become a key research area in network security.

Network traffic in real-world scenarios is diverse and complex. As shown in Figure 1, the detection method will encounter known, variant¹ and unknown malicious traffic when running in a real environment, which requires the method to be robust and reliable. Firstly, they must be able to accurately distinguish known normal and malicious traffic while reducing false negatives and false positives. Secondly, they need to exhibit a certain degree of generalizability, enabling them to recognize variant and unknown malicious traffic. Thirdly, the detection process should be credible to ensure its reliable application in the real world. While ensuring complete accuracy is challenging, the methods should at least inform users when encountering traffic that they cannot adequately address.

Machine learning and deep learning based malicious traffic detection methods have been widely developed over the past few decades. They detect malicious traffic using statistical information, sequence information or raw packets through supervised or unsupervised approaches. However, they are unable to fully fulfill the aforementioned requirements. Current supervised methods [2], [3], [4], [5], [6], [7], [8], [9] can accurately detect known malicious traffic and possess a certain degree of generalization, but are ineffective in identifying unknown attacks. When the distribution of training and test data differs, their performances decline severely. More critically, as shown in Figure 2, the failures are silent. The model is over-confident and does not recognize whether they are capable of processing a given input. The silent misclassification can leave systems in an unnoticed risk state, making network administrators unable to trust these methods and hindering their application in the real world. Current unsupervised methods [10], [11], [12], [13] can detect emerging threats by identifying traffic that deviates from past normal behaviors or the majority as malicious traffic. However, the essence of unsupervised methods is detecting anomalies rather than maliciousness specifically. When the difference between normal and malicious traffic is not obvious, they may produce

¹Variant attacks can be considered as unknown attacks to some extent, so in the following, we use the concept of unknown attacks interchangeably for convenience and simplicity. In some cases, unknown attacks include both variant attacks and truly unknown attacks.

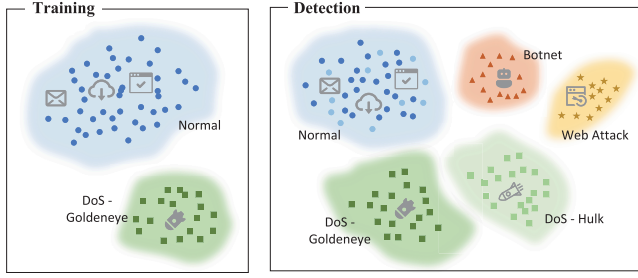


Fig. 1. Conceptual diagram of network traffic types during model training and detection. During training, the model is exposed to normal traffic and several specific types of malicious traffic (DoS-GoldenEye). However, in the deployment of the model in real-world scenarios, not only known malicious traffic (DoS-GoldenEye) is encountered, but also variants of these types (DoS-Hulk) and previously unknown malicious traffic (such as Botnet). These diverse and complex malicious traffic pose challenges for network security.

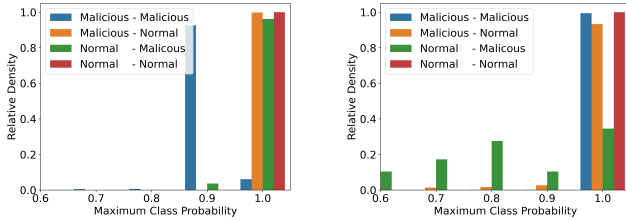


Fig. 2. The distribution of maximum class probability (MCP) of a supervised method, GTID [7], on the CICIDS2018-III dataset (left) and the EnTra2023-III dataset (right). “Normal - Malicious” indicates that normal traffic is classified as malicious. The sum of the relative densities of bars of each color equals 1. It can be seen a large portion of misclassified normal traffic (green bars) and malicious traffic (orange bars) have MCP close to 1, indicating that GTID is very confident about these results. So, users are unable to perceive potential misclassifications by observing MCP, that’s to say, the failures are silent.

a high false positive rate. Besides, most unsupervised methods are based solely on normal or unlabeled traffic, and cannot directly utilize the resources of existing attack data, putting them at a disadvantage in detecting known and variant attacks.

Furthermore, there are many methods that focus on achieving robust malicious traffic detection from various perspectives, yet they still encounter several problems. Extracting more robust features [14], [15], [16] that are less susceptible to various changes is a common way to maintain the model’s good performance in changing environments. Nevertheless, their utility is limited to scenarios where only minor changes in traffic occur, such as bandwidth fluctuations. When facing unknown or variant malicious traffic, they still have the problems of misclassification and overconfidence as previously discussed. Other methods [17], [18] divide the malicious traffic detection problem into multiple stages, progressively performing unknown traffic detection and traffic classification to robustly detect complex malicious traffic, including both known and unknown types. However, the separation of multiple stages may prevent achieving a globally optimal solution, and errors from the initial stage may propagate to the subsequent stages.

To solve these problems, we propose a novel end-to-end method ECNet, which achieves accurate, robust, and reliable detection of both known and unknown malicious traffic. ECNet mainly consists of two parts, one extracting comprehensive multi-view features of network traffic and the

other utilizing a confidence mechanism for model training and malicious traffic detection. Firstly, we extract content features using PacketCNN and attention mechanisms based on randomly selected raw packets, and pattern features through a unique coding method that encodes packet length and direction sequences, followed by SessionCNN. A gated-based feature fusion method is also employed to integrate these features flexibly. The comprehensive dimensions of features and advanced extraction method ensure the accuracy of ECNet, while the random packet selection and the applications of CNN and Transformer grant ECNet generalization ability. Secondly, we innovatively introduce a confidence mechanism and integrate it into model training and malicious traffic detection. For each session, ECNet not only outputs the probability that it belongs to each category but also provides a confidence value, which indicates the reliability of the result. In a normally functioning network, the normal traffic does not exhibit significant changes over a period of time. Based on this, we identify traffic with low confidence as anomalous, which enables the detection of unknown malicious traffic. For known malicious traffic, it typically has higher confidence and can be accurately classified through the maximum category probability. Overall, ECNet integrates the advantages of both supervised and unsupervised learning. It leverages labeled data for training, facilitating the accurate and confident detection of known malicious traffic. It employs confidence values to evaluate the model’s degree of certainty with samples, thus enabling users to trust the outcomes and to identify unknown malicious traffic. Besides, ECNet adopts an end-to-end approach that offers a new unified paradigm for the detection of complex malicious traffic, demonstrating more significant advantages compared to other multi-stage methods.

In conclusion, the main contributions of this paper include:

- We propose ECNet, an end-to-end robust malicious network traffic detection method. It incorporates multi-view feature extraction and a confidence mechanism, and can detect both known and unknown malicious traffic.
- We develop comprehensive features from multi-view to fully represent traffic, including content and pattern features, and design a gated-based method for feature fusion, ensuring the accuracy and generalizability of ECNet.
- We introduce a confidence mechanism that can assess the certainty of the model’s results, and design a probability branch and a confidence branch for ECNet. By integrating these branches, ECNet can achieve accurate and reliable detection of known and unknown malicious traffic.
- We conduct comprehensive experiments on six reorganized datasets that can reflect the complexity of malicious traffic in the real world. The results demonstrate that ECNet is superior to others. It improves F1 by 22.41% compared to supervised methods and 14.15% to unsupervised methods in detecting unknown malicious traffic.

II. RELATED WORK

A. Supervised Malicious Network Traffic Detection

Supervised malicious network traffic detection methods rely on pre-labeled data to train models, learning the features of

normal and malicious traffic to distinguish between them. Depending on the models used, they can be categorized into machine learning based and deep learning based methods.

Machine learning based methods first extract statistical features from network traffic, such as duration and packet count, and then train machine learning models like SVM and Decision Trees based on these features. Gu et al. [2] used Naive Bayes for feature embedding to obtain high-quality features and then constructed a detection model with SVM. Anderson and McGrew [3] extracted over 300 standard and enhanced features from sessions, including metadata and byte distribution, and used a Random Forest as a detector. These methods are computationally efficient and easy to implement, but the selection of features depends on expert knowledge, and they are unable to capture complex malicious traffic patterns.

Deep learning based methods can solve the problem of manual dependency in feature selection and improve detection accuracy. They use CNNs, LSTMs, Transformers, and GNNs to automatically learn latent features from sessions, such as spatial, temporal and topological features. They mainly use the sequential information of traffic or directly use raw traffic as input.

Sequential information refers to a series of feature sequences extracted from network traffic, such as packet length sequences and packet direction sequences. They retain the temporal order information of traffic and can capture dynamic behavior patterns in network communications. The packet length sequence is one of the most commonly used features. Liu et al. [6] proposed an end-to-end detection model called FS-Net. It introduced a reconstruction mechanism and employed a GRU-based autoencoder to learn more distinctive features from the packet length sequence for classification. Hu et al. [19] modeled the packet length sequence and direction sequence of a session as a graph. Then, they utilized Graph2Vec to learn the representation of the graph.

Both statistical and sequential features involve some information loss during the feature extraction process. Raw traffic provides complete information about network traffic, allowing for a fine-grained analysis of packets. Yu et al. [5] used two CNNs to successively learn packet-level and session-level features of network traffic. They also used few-shot learning to improve the detection reliability of minor categories. Zhang et al. [8] encoded each packet into a byte-level traffic graph, and then utilized GNN to get an overall representation vector for each packet. Han et al. [7] extracted packet-level features through n-gram and proposed a time-aware Transformer to extract session-level features for classification. Using raw network traffic can yield good performance, but it requires more computational and storage resources.

These supervised methods show extremely high performance in detecting known malicious traffic. However, their performance significantly decreases when facing unknown attacks, along with issues of silent misclassification.

B. Unsupervised Malicious Network Traffic Detection

Unsupervised malicious network traffic detection methods do not rely on pre-labeled data and can detect unknown

attacks. They primarily detect malicious traffic by identifying patterns that deviate from past normal traffic behaviors or the majority of traffic.

Constructing a model that can perfectly reconstruct normal traffic and considering traffic that cannot be reconstructed as malicious is a common method. The autoencoder is the most commonly used model for this purpose. Mirsky et al. [10] proposed an online plug-and-play method called Kitsune. It maintained a time window to dynamically extract the contextual features of traffic, then trained an ensemble of autoencoders to reconstruct these features, thereby learning the patterns of normal traffic. Catillo et al. [11] proposed CPS-Guard, which used an autoencoder to reconstruct session features. They also designed an outlier-aware algorithm to adaptively select thresholds for distinguishing normal and malicious traffic.

Traffic that deviates significantly from the majority can be considered malicious, and this can be achieved through clustering algorithms or isolation forests. Zhang et al. [13] employed isolation forest to detect isolated malicious traffic. They also proposed an on-demand evolution isolation forest to enable the system to detect malicious traffic in real-time and adapt to environmental changes. Fu et al. [12] constructed network traffic into a flow interaction graph, and then sequentially identified abnormal components and nodes using DBSCAN, ultimately pinpointing the malicious traffic. In this process, they analyzed the connectivity, sparsity, and statistical features of the graph.

Unsupervised methods can detect unknown attacks, but their performance generally does not reach the level of supervised methods. Some malicious traffic has minimal differences from normal traffic, making it difficult for unsupervised methods to distinguish them, whereas supervised methods can accurately identify them using existing knowledge. Additionally, unsupervised learning methods tend to have a higher false positive rate for normal traffic.

C. Robust Malicious Network Traffic Detection

Conventional malicious network traffic detection methods often experience performance degradation when faced with variations in traffic. Consequently, there are some studies that focus on robust malicious network traffic detection.

Most studies aim to design more robust features that remain stable even when traffic changes due to environmental factors or other conditions. Diallo and Patras [14] used cluster centers as an extension of traffic features to address the challenge of sensitivity to subtle changes in traffic features. Fu et al. [16] extracted frequency domain features from the per-packet feature sequences and then used statistical clustering to detect malicious traffic. They claimed that frequency domain features can represent fine-grained information and are not disturbed by the injected noise packets, enabling accurate and robust detection. Xie et al. [15] thoroughly analyzed the TCP mechanisms that may affect packet length sequences of flows, and proposed a self-supervised learning-based traffic augmentation mechanism capable of generating effective feature vectors that robustly represent network traffic in diverse network environments. However, these methods still struggle to effectively

detect unknown malicious traffic that has not previously been encountered during training.

Some methods can simultaneously identify both known and unknown malicious traffic robustly. Yang et al. [17] formulated the problem as a two-stage minimization problem. They first used Conditional Variational Auto-Encoder (CVAE) to classify known malicious traffic by minimizing empirical risk, then utilized Extreme Value Theory to model the distribution of reconstruction errors from the CVAE and identify unknown attacks based on abnormal reconstruction errors. Qiu et al. [18] also followed a similar staged approach. OpenMax was first used to detect some unknown malicious traffic, then they identified unknown malicious traffic that has been misclassified into known categories based on reconstruction loss. This multi-staged approach involves independent training for each stage, which compared to end-to-end methods, struggles to achieve a globally optimal solution. When separately carrying out anomaly detection and traffic classification, it encounters issues common to conventional methods, such as the silent misclassification of supervised methods and the high false positive rates of unsupervised methods. Moreover, errors in earlier stages can adversely affect subsequent stages.

III. THREAT MODEL AND DESIGN GOAL

Our objective is to propose a robust malicious network traffic detection method that can efficiently and accurately detect traffic generated by known and emerging attacks. We use sessions reassembled from traffic based on five-tuple (source IP, source port, destination IP, destination port, protocol) as inputs, and it contains bidirectional packets. Our approach can be deployed at the network edge to analyze traffic between devices in the local area network (LAN) and external Internet, working in conjunction with firewalls and intrusion detection systems. Additionally, our method can also monitor network traffic between hosts within a LAN through the mirror ports of switches.

The main goals of network attackers include disrupting the availability of devices, stealing or altering sensitive information, and using compromised devices to conduct other malicious activities. To achieve these goals, attackers may employ external devices to perform attacks, such as brute force, DDoS, and C&C communications. They can also leverage devices within the LAN that have already been compromised to initiate attacks like DoS, network scanning, and lateral movements against other devices. Our method only focuses on detecting active attacks that generate network traffic and does not consider passive attacks that do not generate traffic, such as privilege escalation [20], passive traffic analysis [21], or local exploitation of vulnerabilities.

IV. METHOD

A. Method Overview

The overall structure of ECNet is shown in Figure 3. ECNet consists of two parts, namely *Multi-view Session Feature Extraction* and *Confidence-based Training and Detection*. Firstly, ECNet learns sufficiently representative features from content and pattern perspectives. Then, the extracted feature

Algorithm 1 Training ECNet

Input: Session S , label y

Output: Well-trained ECNet

```

1: Initialize the parameters of ECNet
2: for epoch do
3:   /* Content feature extraction */                                ▷ IV-B1
4:   Randomly select  $n_p$  packets from  $S$ 
5:   Perform content augmentation on packets                        ▷ IV-C3
6:   Extract content feature  $f_c$  of  $S$  from augmented packets

7:   /* Pattern feature extraction */                                ▷ IV-B2
8:   Get length sequence  $L$  and direction sequence  $D$  of  $S$ 
9:   Encode  $L$  and  $D$  into  $v_{sq}$ 
10:  Perform pattern augmentation on  $v_{sq}$                           ▷ IV-C3
11:  Extract pattern feature  $f_e$  of  $S$  from augmented  $v_{sq}$ 
12:  /* Gated-based feature fusion */                                ▷ IV-B3
13:  Fusion  $f_c$  and  $f_e$  to get overall representation  $f_s$  of  $S$ 
14:  /* Probability and Confidence generator */                       ▷ IV-C1
15:  Get the probability  $p$  and confidence  $c$ 
16:  /* Model training */                                           ▷ IV-C2
17:  Calculate loss  $\mathcal{L}$  and update the parameters of ECNet
18: end for
19: return Well-trained ECNet

```

Algorithm 2 Detecting Malicious Traffic With ECNet

Input: Session S , Well-trained ECNet

Output: prediction r

```

1: /* Content feature extraction */                                ▷ IV-B1
2: Randomly select  $n_p$  packets from  $S$ 
3: Extract content feature  $f_c$  of  $S$  from packets
4: /* Pattern feature extraction */                                ▷ IV-B2
5: Get length sequence  $L$  and direction sequence  $D$  of  $S$ 
6: Encode  $L$  and  $D$  into  $v_{sq}$ 
7: Extract pattern feature  $f_e$  of  $S$  from  $v_{sq}$ 
8: /* Gated-based feature fusion */                                ▷ IV-B3
9: Fusion  $f_c$  and  $f_e$  to get overall representation  $f_s$  of  $S$ 
10: /* Probability and Confidence generator */                       ▷ IV-C1
11: Get the probability  $p$  and confidence  $c$ 
12: /* Malicious traffic detection */                                ▷ IV-C4
13: Get predicted result  $r$ 
14: return  $r$ 

```

is used for model training and malicious traffic detection, and a confidence mechanism is introduced in these processes. Algorithm 1 and Algorithm 2 show the processes of training ECNet and using ECNet to detect malicious traffic, respectively.

B. Multi-View Session Feature Extraction

To learn comprehensive session features that are representative enough to distinguish normal and malicious traffic, we extract features from both the content of the session's transmissions and the interaction patterns between the communicating parties. Specifically, we randomly select a small subset of packets from each session to learn content features. At the same time, we extract the packet length and direction

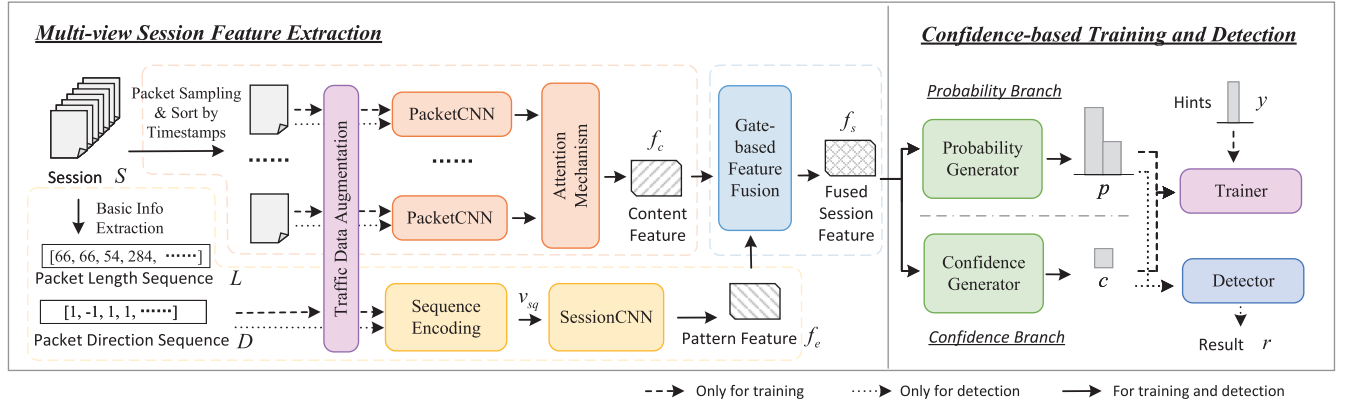


Fig. 3. The overall structure of ECNet. Firstly, ECNet separately extracts content features and pattern features from each session, and then uses a gated-based feature fusion method to combine these features in the multi-view session feature extraction module. Next, the fused session features are used for model training and malicious traffic detection. During training, traffic data augmentation is first performed on the sessions in training data. The session features extracted from the augmented data are the inputs for the probability generator and confidence generator, resulting in p and c . At the same time, hints y also need to be input into the trainer. During detection, data augmentation is not required. The results are obtained using p and c .

sequences of the session to learn pattern features from them. We also design a gate-based feature fusion method to appropriately fuse these features together. Traffic data augmentation is designed to optimize the training process, so it will be introduced in detail in Section IV-C.

1) *Content Feature Extraction*: Raw packets carry rich information, including header fields and payload, and can better characterize network behaviors. They are useful for detecting attacks with unusual signatures or attacks that deliver malicious payload. Besides, it is highly challenging for attackers to evade detection by mimicking normal traffic through modifying the raw packets while ensuring the quality of the attack. Therefore, we choose to learn content features from raw packets.

Although raw packets are useful in detecting malicious traffic, it is noteworthy that different packets in a session often contain much overlapping information. For example, TCP traffic can be divided into those generated during the connection establishment phase and those generated during the data transmission phase.

So, it is unnecessary to use all available packets. Employing all packets can result in increased storage and computational overhead, and introduce noise. Besides, since we subsequently extract pattern features from other perspective, we only need to obtain the transmitted content information from the raw data packets. Based on these, we randomly select n_p packets from a session and sort them by timestamps to learn content features. In this paper, we set n_p to 4 as a choice balancing performance and complexity. We aim to have the randomly selected packets include those from both forward and backward connection establishment and data transmission processes. Setting n_p to 4 ideally allows for the inclusion of packets representing these scenarios.

For each packet, IP and MAC addresses are first removed. Then, we convert the hexadecimal packets into decimal format and represent each packet as a one-dimensional vector of size 200, denoted as f_p . If the packet length exceeds 200, the excess is removed, and if it is less than 200, zero vector is

padded. To learn the latent feature of each packet, we design a PacketCNN. It consists of three layers of 1d-CNN and two layers of linear layers. The process is formulated as Equation (1), and f'_p is the extracted packet feature and $F_{PC}(\cdot)$ denotes the PacketCNN.

$$f'_p = F_{PC}(f_p) \quad (1)$$

For different types of attacks or normal behaviors, different packets play distinct roles. For example, in SQL injection attacks, the data packets during transmission are often abnormal, whereas in flooding attacks, anomalies typically arise during the handshake phase. To enable the model to focus on the most crucial packets, we employ a multi-head attention mechanism to extract content features based on the extracted packet features.

The process of the attention mechanism is as show in Equation (2). The query, key, and value vectors in this paper are calculated as $Q = f_P W_Q$, $K = f_P W_K$ and $V = f_P W_V$. $f_P = [f_p^1, \dots, f_p^{n_p}]$ is a vector consisting of packet features, and we sort packet features by timestamp to align different sessions. W_Q , W_K and W_V are learnable weights.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (2)$$

The multi-head attention mechanism allows the model to focus on information at different levels, enhancing both the model's expressive power and robustness. The process is formulated as follows, and $head_i$ is the attention value of i^{th} head, W_h^Q , W_h^K , W_h^V and W_h are learnable weights, $concat(\cdot)$ denote the concatenate operation.

$$MultiAtt = concat(head_0, \dots, head_3)W_h \quad (3)$$

$$head_i = Attention(QW_h^Q, KW_h^K, VW_h^V) \quad (4)$$

Then, the multi-head attention values are averaged to obtain the content feature f_c of the session.

$$f_c = \frac{1}{n_p} \sum_{i=1}^{n_p} MultiAtt_i \quad (5)$$

Algorithm 3 Sequence Encoding**Input:** $L = (l_1, \dots, l_N)$, $D = (d_1, \dots, d_N)$, m **Output:** v_{sq}

```

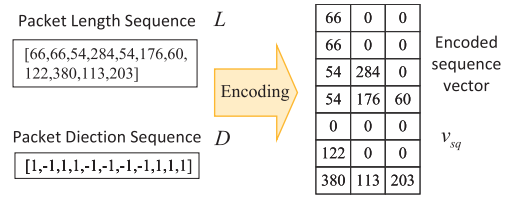
1: Initialize  $v_{sq}$  and  $v_{tmp}$  as empty vectors
2: for  $l_i \in L$ ,  $d_i \in D$  do
3:   if  $d_i$  equals to  $d_{i-1}$  then
4:     if  $\text{len}(v_{tmp})$  is divisible by  $m$  then
5:       Append  $m$  zeros to  $v_{tmp}$ 
6:     end if
7:     Append  $l_i$  to  $v_{tmp}$ 
8:   else
9:     Reshape  $v_{tmp}$  into a 2D vector of  $m$  columns
10:    Append  $v_{tmp}$  to  $v_{sq}$ , and set  $v_{tmp}$  as an empty vector
11:    Append  $l_i$  to  $v_{tmp}$ 
12:   end if
13: end for
14: Reshape  $v_{tmp}$  into a 2D vector and append it to  $v_{sq}$ 
15: return  $v_{sq}$ 

```

2) *Pattern Feature Extraction*: For malicious traffic, besides anomalies in content, there can also be abnormalities in behavioral patterns. For instance, an attack might generate a large amount of burst traffic or maintain long-term regular traffic. Besides, due to the shared use of attack tools and methodologies, similar types of attacks may exhibit similar behavior pattern [22]. Therefore, learning the behavioral patterns of sessions is vital for detecting malicious traffic.

To learn the pattern feature of a session, we first extract the length and direction of packets in a session, and combine them as two sequences, noted as $L = (l_1, \dots, l_N)$ and $D = (d_1, \dots, d_N)$. $l_i \in \mathbb{N}$ is the length of i^{th} packet. $d_i \in \{1, -1\}$ is the direction of i^{th} packet. Then, we encode these two sequences into a two-dimensional vector, noted as v_{sq} . The process of sequence encoding is depicted in Algorithm 3, and Figure 4 shows an example of sequence encoding. m is the number of columns for v_{sq} , and we set m to 3. For normal traffic, aside from large file transfers and streaming services, it is rare to encounter many consecutive packets in the same direction. We use burst to represent a series of contiguous packets sent in the same direction [23]. Bursts with a size of less than or equal to 3 accounts for 80% of all cases, therefore setting m to 3 maximizes the efficiency of our encoding method and avoids creating too much redundant data.

Our encoding method can effectively and robustly represent the communication pattern within a session. Firstly, the encoded vector represents the direction well without using signs. Forward and backward directions are relative concepts, and directly using the signs can introduce noise when source and destination hosts change. Our method is robust in this situation and has better transferability across different environments. Secondly, our method can capture the sequence, frequency, and burst of the content transmitted. Packet length represents the content to some extent. Each burst appears in

Fig. 4. An example of sequence encoding ($m = 3$).

one row, a burst with many packets is represented as adjacent consecutive rows, and the initiation of each burst aligns in the same column.

After obtaining v_{sq} , we design a SessionCNN based on CNN to learn pattern features. The spatial awareness of the CNN structure aids in extracting features of traffic displayed on the rows and columns of v_{sq} . SessionCNN consists of two layers of 2d-CNN and two layers of linear layer. As illustrated in Equation (6), $F_{SC}(\cdot)$ is SessionCNN, and f_e denotes the pattern feature.

$$f_e = F_{SC}(v_{sq}) \quad (6)$$

3) *Gated-Based Feature Fusion*: The content feature and pattern feature of a session are not mutually independent. We propose a gated-based feature fusion method to integrate them to get an overall representation of the session. The gating mechanism provides a flexible, effective, and robust method for feature fusion.

As shown in Equation (7) and (8), we first get two gate vectors g_c and g_e . The content feature f_c and pattern feature f_e are initially input to a bilinear function to obtain a temporary vector, and then each element of the vector is scaled to $[0, 1]$ by a sigmoid function σ . W_{cg} , W_{eg} , b_{cg} and b_{eg} are learnable weights and biases. These two gate functions do not share parameters, thus considering different perspectives.

$$g_c = \sigma(f_c^T W_{cg} f_e + b_{cg}) \quad (7)$$

$$g_e = \sigma(f_e^T W_{eg} f_c + b_{eg}) \quad (8)$$

Then, f_c and f_e are element-wise multiplied with their respective gate vectors and are concatenated into the final fused session feature f_s .

$$f_s = \text{concat}(g_c \odot f_c, g_e \odot f_e) \quad (9)$$

C. Confidence-Based Training and Detection

Existing methods cannot handle unknown malicious traffic well and are prone to false negatives. They also tend to be overly confident and assign high maximum category probabilities to their outcomes, leading silent misclassification. These uncontrolled failures can damage user's trust in the method. We introduce a confidence mechanism to solve these problems.

In this section, we first introduce the probability generator and confidence generator, which are shared components for training and detection. We then introduce the details of model training and traffic data augmentation method and finally describe the confidence-based malicious traffic detection method.

1) *Probability and Confidence Generator*: ECNet is different from traditional classifiers. It has two branches: a probability branch and a confidence branch.

In the probability branch, the probability generator is the same as traditional classifiers. The session feature f_s is firstly processed by a fully connected neural network $F_{PG}(\cdot)$, and then the output is converted into the probabilities p of belonging to each category by a softmax function.

$$p = \text{softmax}(F_{PG}(f_s)) \quad (10)$$

In the confidence branch, f_s is also first processed through a fully connected neural network $F_{CG}(\cdot)$ to obtain a value, then it is scaled between 0 and 1 using a sigmoid function to obtain the confidence value c .

$$c = \sigma(F_{CG}(f_s)) \quad (11)$$

Both F_{PG} and F_{CG} are composed of two linear layers and a dropout layer. The difference is that the last layer of F_{PG} has two neurons, representing normal and malicious, while F_{CG} has only one neuron. We use two independent neural networks to process f_s separately instead of using one network directly. This is because the goals of obtaining probabilities and confidence values are different. Employing a single neural network necessitates a sufficiently complex network structure and parameters to capture all the features and relationships required for both targets. Such complexity increases the training burden and affects model convergence.

2) *Model Training*: Inspired by Devries's work [24] on OOD image classification, we correlate the model's confidence with the hints needed in its decision-making process. The model is allowed to ask for some hints, that is true labels, to assist in decision-making when facing inputs that it is uncertain about or unfamiliar with, but there is a certain penalty for using hints. After full training, the model will not request hints for inputs it is very certain about or has seen before, where its confidence is high. For unfamiliar inputs, it will request real labels as hints, indicating lower confidence.

Based on these, the probabilities after using hints are calculated as Equation (12), where p_i and y_i are the predicted and real probabilities of the session belonging to i^{th} category respectively. c is the confidence generated by the confidence generator. The closer c is to 1, the more confident the model is in its prediction results.

$$p'_i = c \cdot p_i + (1 - c) \cdot y_i \quad (12)$$

The classification loss $\mathcal{L}_p$ using modified probabilities is calculated as follows. n_c is the number of categories and it equals 2.

$$\mathcal{L}_p = \sum_{i=1}^{n_c} -y_i \log(p'_i) \quad (13)$$

To prevent the model from always setting c to 0 for using real labels to make decisions, we add a confidence loss $\mathcal{L}_c$ as the penalty of using hints.

$$\mathcal{L}_c = -\log(c) \quad (14)$$

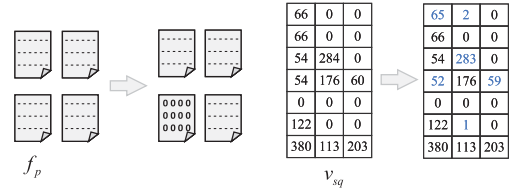


Fig. 5. An example of traffic data augmentation for a session. The left shows the augmentation for content, and the right for pattern.

The overall loss $\mathcal{L}$ is the sum of prediction loss and confidence loss. λ is the weight of confidence loss, and it is a hyper-parameter to balance these two losses.

$$\mathcal{L} = \mathcal{L}_p + \lambda \mathcal{L}_c \quad (15)$$

In the early stage of training, capturing good initial parameters is crucial as it facilitates faster convergence and improves the final performance of the model. If the model is allowed to use hints with a low penalty in the early stage, it will hamper the learning of features used for classification in the probability branch. Conversely, in the later stage, after the model has been exposed to a sufficient number of samples, it should focus more on enhancing the confidence of its predictions. Therefore, we set λ to be gradually decreasing, which means the cost of using hints reduces gradually [25]. The decay process of λ with the training step s is shown in Equation (16). λ_0 is the initial value of λ , β is used to adjust the decay speed.

$$\lambda(s) = \lambda_0 * e^{-s/\beta} \quad (16)$$

We also follow Devries [24] to give hints to the 50% of the inputs to prevent the model from ignoring outliers.

3) *Traffic Data Augmentation*: When over-fitting occurs, the model may accurately predict all inputs with a confidence of 1. It will lead to the failures of the confidence branch when facing test data. To address this problem, we specially design a traffic data augmentation method to generate hard samples.

Data augmentation is performed from both content and pattern perspectives. For content augmentation, we randomly set one of the n_p packet features f_p used in content feature extraction to zero vectors. For pattern augmentation, we add random perturbations to the encoded sequence vector v_{sq} to modify the communication pattern of the session. Specifically, we randomly pick half the rows from v_{sq} and randomly add or subtract 1 to one or two values in each row. If the altered value becomes negative, it is maintained at 0. Figure 5 illustrates an example of content augmentation and pattern augmentation.

We perform content augmentation on the data comprising a proportion of α_c , and pattern augmentation on the data comprising a proportion of α_p . Note that a session might undergo both content augmentation and pattern augmentation simultaneously.

4) *Malicious Traffic Detection*: A well-trained ECNet can provide low confidence for unknown data while giving high confidence and accurate class probabilities for inputs it is certain about. Once we have trained a model, we can use it to perform malicious traffic detection. In a normally functioning network environment, the traffic generated by users' routine activities usually does not change significantly. Traffic that

differs greatly from past patterns in a network can be generally considered to be malicious traffic.

Based on these observations, we can detect malicious traffic as shown in Equation (17). r is the final predicted category to which the test data belongs. 0 represents normal and 1 represents malicious. If the confidence c is lower than the threshold δ_c , it means that the model has not previously processed related data, then the traffic is classified as malicious. This approach enables the detection of unknown malicious traffic. Conversely, if the confidence is higher than the threshold, the class with the higher probability is assigned as the sample's category, enabling detection of known malicious traffic.

$$r = \begin{cases} 1, & c < \delta_c \\ \arg \max_{i \in \{0,1\}} p'_i, & \text{otherwise} \end{cases} \quad (17)$$

The confidence values for known normal and malicious traffic are close to 1, and they rely on the category probabilities for classification. The confidence values for unknown malicious traffic are relatively low, and they are directly classified as malicious. At this point, the probabilities are untrustworthy and meaningless.

V. EVALUATION

A. Experiment Overview

To thoroughly validate the performance of ECNet, we conduct comprehensive experiments. We first introduce the experimental setup, including the datasets, baseline methods, and evaluation metrics. Specifically, to reflect the complex situation of traffic in the real world, we reorganize the datasets. Then, we detail the experimental results, comparing the performance of ECNet and baselines in detecting both known and unknown malicious traffic, and also comparing their complexities. The results indicate that our method outperforms other methods, especially in detecting variants and unknown malicious traffic. Additionally, we perform ablation studies and sensitivity analysis of parameters on ECNet for a deeper understanding of its workings.

B. Experiment Setup

1) *Datasets*: Randomly splitting data into training and test sets fails to reflect the diverse traffic in the real world. To verify the performance comprehensively, we use two basic datasets and reorganize them for evaluation.

a) *Basic datasets*: We use the public dataset CICIDS2018 [26] and the self-collected dataset EnTra2023 as basic datasets.

- *CICIDS2018*. The CICIDS2018 dataset is a widely used dataset for malicious network traffic detection. It simulates an enterprise network environment, with approximately 500 devices involved. It encompasses a variety of attacks such as Brute Force, Botnet, DoS, DDoS, and Web Attacks, alongside normal activities.
- *EnTra2023*. The EnTra2023 dataset is collected by ourselves. Its normal traffic comes from a day's network activity of select hosts within an enterprise network. The malicious traffic is sourced from the Malware Capture

Facility Project (MCFP). From this project, we gather traffic generated by spyware, miner, ransomware, botware, and adware, totaling 64 pcap files.

b) *Data reorganization*: In real-world scenarios, the malicious traffic may generated by known, variant and unknown attacks. We reorganize the two datasets accordingly. Table I presents the reorganization details of malicious traffic.

For the scenario of known attacks, we randomly mix all traffic in each dataset, including normal and malicious traffic, and then split them into two parts for the training and testing sets. The new datasets reorganized in this manner are named *CICIDS2018-I* and *EnTra2023-I*.

Variant attacks refer to attacks belonging to the same category, but they have adjustments in strategies, methods, or other characteristics. For the CICIDS2018 dataset, we place similar attacks using different tools into the training and testing sets, such as DoS-GoldenEye and DoS-Slowloris. We use the normal traffic of the first five days as training data and the last three days as test data. For the EnTra2023 dataset, traffic from different pcap files is allocated to the training and testing sets separately. The normal traffic is also divided into training and testing sets based on the sequence of time. The two datasets reorganized in this way are named *CICIDS2018-II* and *EnTra2023-II*. The normal traffic is divided into training and testing sets based on the sequence of time. The two reorganized datasets are *CICIDS2018-II* and *EnTra2023-II*.

For unknown attacks, we put different types of attacks into the training and testing sets for the CICIDS2018 dataset. For the EnTra2023 dataset, the training set only contains traffic generated by three types of malware, while the testing set includes traffic from other categories or types of malware. The division of normal traffic follows the same methodology as the variant attack datasets. The reorganized datasets are *CICIDS2018-III* and *EnTra2023-III*.

For each dataset, we randomly selected 10% of the data from the training set to serve as the validation set for hyperparameter tuning and model selection.

c) *Statistics of datasets*: To ensure the comparability of results across the three reorganized datasets and to balance the ratio of normal and malicious traffic, we sample the datasets, and the statistical information is shown in Table II.

2) *Baselines*: To thoroughly evaluate the performance of ECNet, we establish seven baseline methods for comparison, including supervised methods (FS-Net [6], CICAUTOML [27], GTID [7]) and an unsupervised method (Kitsune [10]), as well as methods specifically focused on robust malicious traffic detection (Rosetta [15], ACID [14], CVAE-EVT [17]).

- *FS-Net*. FS-Net can learn representative features from packet length sequences. It utilizes a multi-layer encoder-decoder structure and reconstruction mechanism to mine the potential sequential characteristics.
- *CICAUTOML*. We combine CIC features with Automated Machine Learning (AutoML) [28] to implement CICAUTOML to represent these ML-based methods as a baseline. CIC features are extracted by CICFlowMeter [29] and are a set of widely used statistical features of network traffic, including more than 80 features.

TABLE I
REORGANIZATION DETAILS OF MALICIOUS TRAFFIC OF THE CICIDS2018 DATASET AND THE ENTRA2023 DATASET

Dataset		Train	Test
CICIDS2018	I	Bruteforce, DoS, Web Attack, Botnet, DDoS	
	II	Bruteforce (FTP Patator), DoS (GoldenEye, Hulk), Web Attack (Brute Force, SQL Injection), Botnet, DDoS (LOIC for HTTP)	Bruteforce (SSH Patator), DoS (Slowloris, Slowhttptest), Web Attack (XSS), DDoS (LOIC for UDP)
	III	Bruteforce (FTP Patator, SSH Patator), DoS (GoldenEye, Hulk, Slowloris, Slowhttptest), Botnet	Web Attack (Brute Force, SQL Injection, XSS), DDoS (LOIC for HTTP, LOIC for UDP)
EnTra2023	I	Spyware, Miner, Ransomware, Botware, Adware	
	II	Spyware (Zesus, Trickster, Magic, CCleaner), Miner (CoinMiner, BitcoinMiner, MinerTrojan), Ransomware (Dirdex, WannaCry), Botware (TrickBot, Emotet, Sality, Sonjan), Adware (AdLoad, WebComp)	Spyware (Zesus, CCleaner), Miner (CoinMiner), Ransomware (Dirdex, WannaCry), Botware (TrickBot, Emotet)
	III	Spyware (Zesus), Ransomware (Dirdex), Botware (TrickBot)	Spyware (Trickster, CCleaner), Miner (CoinMiner, BitcoinMiner, MinerTrojan), Ransomware (WannaCry), Adware (AdLoad, WebComp), Botware (Emotet, Sality, Sonjan)

TABLE II
THE STATISTICS OF DATASETS

Dataset		Train & Validate		Test	
		Normal	Malicious	Normal	Malicious
CICIDS2018	I	630,000	157,618	270,000	67,550
	II	600,000	159,499	300,000	65,669
	III	600,000	202,789	300,000	32,379
EnTra2023	I	1,813,623	354,749	777,284	152,036
	II	1,813,647	331,740	777,300	175,045
	III	1,813,647	324,810	777,300	181,975

- *GTID*. GTID utilizes raw network traffic directly for analysis. It uses n-gram frequency to extract packet-level features, and uses a time-aware Transformer to extract session-level features for malicious traffic detection.
- *Kitsune*. Kitsune is an unsupervised malicious traffic detection method. It uses an ensemble of autoencoders to reconstruct the features of normal traffic, and the traffic with high reconstruction error is considered malicious.
- *Rosetta*. Rosetta is a robust traffic classification method in diverse network environments. It leverages TCP-aware traffic augmentation mechanisms and self-supervised learning to generate robust features based on packet length sequences and uses LSTM as the classifier.
- *ACID*. ACID uses a supervised adaptive clustering technique to learn cluster centers to expand the features, thereby enhancing the robustness against outliers and the generalization abilities of the model in the face of evolving traffic landscapes.
- *CVAE-EVT*. CVAE-EVT employs conditional autoencoders and extreme value theory to implement a hierarchical detection method, and performs staged classification of known attacks and detection of unknown attacks.

We also offer a simplified version of ECNet that does not utilize confidence, named ECNet-Simple, for comparison. We introduce it here for ease of reference in the following sections, although it is not a baseline method in the traditional sense.

- *ECNet-Simple*. Its feature extraction component is the same with ECNet. After obtaining the fused session

TABLE III
PERFORMANCE COMPARISON ON THE CICIDS2018-I DATASET (%)

Method	Accuracy	Precision	Recall	F1
FS-Net	99.46	99.18	98.10	98.64
CICAutoML	99.87	99.53	99.86	99.69
GTID	99.96	99.82	99.83	99.83
Kitsune	92.39	75.91	91.01	82.78
Rosetta	99.25	97.84	98.31	98.07
ACID	99.75	99.22	99.57	99.40
CVAE-EVT	99.19	96.95	99.20	98.06
ECNet - Simple	99.89	99.47	99.99	99.71
ECNet	99.98	99.99	99.96	99.98

TABLE IV
PERFORMANCE COMPARISON ON THE ENTRA2023-I DATASET (%)

Method	Accuracy	Precision	Recall	F1
FS-Net	98.01	98.35	96.36	97.35
CICAutoML	99.80	99.28	99.51	99.39
GTID	99.97	99.98	99.95	99.96
Kitsune	82.81	82.49	72.66	77.27
Rosetta	96.78	96.61	94.84	95.72
ACID	99.52	98.81	98.28	98.55
CVAE-EVT	99.67	99.24	98.75	98.99
ECNet-Simple	99.98	99.99	99.97	99.98
ECNet	99.97	99.98	99.99	99.98

features, it employs a fully connected network and a softmax layer for both training and detection.

3) *Metric*: To offer a comprehensive view of the model's performance across various aspects of predictive accuracy and reliability, we choose Accuracy, Precision, Recall and F1 score as our evaluation metrics.

C. Known Malicious Traffic Detection

We conduct experiments of known malicious traffic detection on the CICIDS2018-I dataset and the EnTra2023-I dataset. Table III and Table IV compare the performance of ECNet and baselines. ECNet achieves an accuracy of 99.97% and an F1 of 99.98% on both datasets, outperforming other baseline methods, and ECNet-Simple also reaches F1 of 99.71% and

TABLE V
PERFORMANCE COMPARISON ON THE CICIDS2018-II DATASET AND THE CICIDS2018-III DATASET (%)

Dataset	CICIDS2018-II				CICIDS2018-III			
Method	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
FS-Net	98.10	90.35	69.73	78.71	90.35	95.25	47.48	63.37
CICAUTOML	87.26	62.36	73.68	67.55	85.75	31.99	40.77	35.85
GTID	96.51	95.18	67.14	78.74	96.71	85.79	41.65	56.08
Kitsune	92.83	73.98	91.31	81.74	92.97	58.55	92.24	71.63
Rosetta	89.86	90.97	46.95	61.93	90.12	47.12	21.96	29.96
ACID	90.98	78.77	68.46	73.17	88.42	41.08	43.36	42.19
CVAE-EVT	96.88	90.01	92.98	91.47	94.48	76.64	62.27	68.71
ECNet-Simple	91.49	96.91	53.28	68.76	93.83	91.89	39.31	55.06
ECNet	98.97	95.83	93.38	94.59	98.44	79.61	93.00	85.78

TABLE VI
PERFORMANCE COMPARISON ON THE ENTRA2023-II DATASET AND THE ENTRA2023-III DATASET (%)

Dataset	EnTra2023-II				EnTra2023-III			
Method	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
FS-Net	98.22	98.28	97.51	97.89	88.41	99.29	72.37	83.72
CICAUTOML	99.42	98.71	98.17	98.44	97.21	98.61	86.48	92.15
GTID	99.78	99.98	99.50	99.74	96.05	99.98	90.43	94.96
Kitsune	83.31	84.58	73.96	78.91	83.34	80.56	78.51	79.53
Rosetta	93.51	94.47	89.88	92.12	87.76	99.17	70.88	82.67
ACID	99.44	98.15	98.84	98.50	97.60	98.24	88.96	93.37
CVAE-EVT	98.21	94.66	96.16	95.41	98.12	94.00	96.22	95.10
ECNet-Simple	99.62	99.98	99.13	99.55	96.85	99.98	92.37	96.03
ECNet	99.82	99.87	99.7	99.79	99.25	99.73	98.44	99.08

99.98%. Their performances are comparable to GTID, indicating that our content and pattern features are representative enough. They can achieve similar performance using all raw network traffic while avoiding noise. ECNet-Simple's performance on both datasets surpasses CICAUTOML, suggesting our features are superior to the statistical features of sessions. For Kitsune, we use normal traffic to train it, and its performance is obviously not as good as other supervised methods, and the precision is lower, indicating that there are many false positives for normal traffic.

Using confidence in ECNet does not significantly enhance performance compared to ECNet-Simple, as the model encounters known patterns during testing and assigns high confidence values to almost all samples, making the confidence branch of ECNet less influential in decision-making.

D. Unknown Malicious Traffic Detection

Detecting unknown malicious traffic is more challenging than detecting known traffic, as it requires the model to have a certain degree of generalization ability. We conduct experiments on CICIDS2018-II, CICIDS2018-III, EnTra2023-II, and EnTra2023-III. We refer to variant attacks and truly unknown attacks as unknown attacks in this section. Variant attacks can be considered unknown to some extent compared to known attacks. Besides, this approach allows us to analyze how these methods perform when faced with varying degrees of differences between training and testing sets.

1) *Overall Performance:* Table V and Table VI compare the performance of ECNet with baselines. It can be seen that ECNet achieves the best performance across all datasets, especially in the case of detecting unknown malicious traffic, where our method significantly outperforms other methods.

For the CICIDS2018-II dataset, our method achieves an accuracy of 98.97% and an F1 score of 94.59%, only a slight decrease from the performance on the CICIDS2018-I dataset (with a decrease of 1.01% in accuracy and 5.39% in F1 score). Other supervised methods show a noticeable decline in performance, especially in terms of recall. For example, FS-Net has a recall of 69.73% and an F1 of 78.71%. The reason for these low recalls is that a lot of malicious traffic is misclassified as normal traffic, which also shows that these methods cannot handle variant malicious traffic well. The recall for Kitsune is 91.31%, which is similar to ECNet's recall, but its precision is lower, indicating that a significant amount of normal traffic is falsely classified as malicious. Furthermore, the performance of robust malicious traffic detection methods that augment features, such as Rosetta and ACID, is unsatisfactory. Their F1s are 61.93% and 73.17%, similar to those of ordinary methods. They can only handle minor changes in traffic, such as those caused by environmental variations, and their robustness in detecting unknown attacks is insufficient. CVAE-EVT performs multi-stage detection of unknown and malicious traffic, achieving an accuracy of 96.88%, which is superior to other methods. However, our ECNet still surpasses it by 2.09%, indicating that our confidence-based end-to-end

approach can achieve a better global solution than the stage-wise method.

For the CICIDS2018-III dataset, due to greater differences between the training and testing data, the baseline methods experience more severe performance drops. GTID's recall is only 41.65%, with an F1 of 56.08%, which is almost the same as random guessing. The F1 of CICAUTOML is 35.85%, even worse than random guesses. In contrast, our ECNet maintains an F1 of 85.78%, a significant 22.41% improvement over the best supervised baseline method, FS-Net. The unsupervised method Kitsune is not significantly affected by the novelty of new attacks since it is trained only on normal traffic, as evident from its accuracy and recall. Comparatively, ECNet achieves a recall similar to that of Kitsune, indicating that our method is capable of reaching a level comparable to unsupervised methods in detecting unknown attacks. The performance degradation of these robust malicious traffic detection methods is also obvious. The F1 of Rosetta and ACID both decrease by approximately 30%, and CVAE-EVT's F1 decreases by 22.76%. In contrast, ECNet's F1 decreases by less than 10%, further demonstrating ECNet's superior robustness in handling unknown traffic compared to other methods.

For the EnTra2023-II dataset, the performance of these methods are still satisfactory. This is because the training and test sets include malicious traffic generated by the same malware. Our ECNet achieves an accuracy of 99.82% and an F1 of 99.79%, surpassing other methods. On the EnTra2023-III dataset, most methods still experience the performance decline observed in the CICIDS2018-III dataset. For example, FS-Net drops to an accuracy of 88.41% and an F1 of 83.72%. The performance of Kitsune on this dataset doesn't change significantly compared to the performance on EnTra2023-II, but is still not as good as supervised methods. When there is a certain degree of correlation between the malicious traffic data in the test set and the training set, Kitsune is unable to leverage this information, which limits its performance. Compared to others, CVAE-EVT and ECNet exhibit minimal performance degradation, but ECNet still significantly outperforms CVAE-EVT, with F1s of 99.08% and 95.10% and accuracies of 99.25% and 98.12%, respectively. ECNet benefits from the rich features extracted from both content and pattern perspectives, as well as from its confidence-based end-to-end learning and detection model, demonstrating excellent capability in detecting unknown malicious traffic. In contrast, CVAE-EVT employs statistical features and adopts a multi-stage learning strategy, resulting in inferior performance compared to ECNet.

2) *Detailed Performance:* To further investigate the performance of methods, we calculate the recall for each type of malicious traffic in the CICIDS2018-III and EnTra2023-III datasets. The recall indicates how much of each category of malicious traffic is correctly identified as such. Figure 6 and Figure 7 display the corresponding heatmaps.

For the CICIDS2018-III dataset, most supervised methods demonstrate decent detection capabilities for DDOS (LOIC for HTTP), likely benefiting from the presence of DoS and Botnet traffic in the training data. However, the recalls for other types of malicious traffic are relatively low. Rosetta, for instance, has a recall of only 2.39% for DDOS (LOIC

FS-Net	51.48	19.54	18.38	26.66	22.64
CICAUTOML	40.72	41.16	41.18	39.57	50.0
GTID	43.49	8.68	17.27	38.33	43.39
Kitsune	92.81	87.9	75.74	45.01	79.25
Rosetta	22.25	2.39	13.6	15.0	13.21
ACID	44.37	29.02	25.05	43.86	52.83
CVAE-EVT	62.17	66.42	68.11	36.6	42.45
ECNet-Simple	41.04	12.71	9.55	10.0	9.43
ECNet	93.0	98.7	79.41	61.67	75.47

Fig. 6. Heatmap of recall for each category of malicious traffic on the CICIDS2018-III dataset.

FS-Net	86.85	76.03	13.56	52.34	86.88
CICAUTOML	87.59	73.05	69.52	87.73	93.35
GTID	72.98	85.01	97.68	77.26	98.07
Kitsune	69.79	67.11	72.01	42.39	90.63
Rosetta	83.84	61.84	9.76	77.26	85.59
ACID	91.98	86.53	93.37	86.86	88.05
CVAE-EVT	92.69	89.64	92.98	81.83	97.56
ECNet-Simple	74.34	96.94	98.08	81.3	98.88
ECNet	97.14	98.86	98.14	98.87	98.96

Fig. 7. Heatmap of recall for each category of malicious traffic on the EnTra2023-III dataset.

for UDP). CICAUTOML's recalls across all categories are relatively uniform but all below 50%. This could be due to a well fit on the training set but a lack of generalization ability, leading to an almost random guess in detecting traffic of all categories. In comparison, ECNet's recall is much higher, reaching 98.7% for DDOS (LOIC for UDP). Its recall for Web attack (XSS) is 61.67%, which might be because the traffic generated by this attack is somewhat similar to normal traffic.

For the EnTra2023-III dataset, ECNet also outperforms other methods in terms of recall across all categories. The lowest recall is 97.14% in detecting spyware. Other baselines have relatively weaker capabilities in detecting malicious traffic generated by adware. The difficulty in detecting adware could be attributed to the significant functional differences between adware and other types of malware, leading to greater variations in traffic patterns. Kitsune exhibits the lowest recall for adware, possibly due to the similarity between adware-generated traffic and normal traffic. In contrast to supervised methods, our ECNet is not limited by whether the same type of malicious traffic appeared in the training set and maintains stable performance in detecting unknown malicious traffic. Compared to unsupervised methods, ECNet can utilize the probability branch to classify data that is similar to normal traffic effectively.

3) *Confidence Visualization:* To analyze the role of the confidence branch in detecting malicious traffic, we visualize the confidence levels of normal and malicious traffic during testing. Figure 8 shows the distribution of confidence

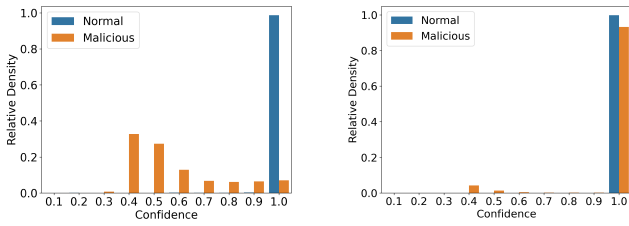


Fig. 8. Confidence distribution of normal and malicious traffic on the CICIDS2018-III dataset (left) and the EnTra2023-III dataset (right).

values on the CICIDS2018-III and EnTra2023-III datasets, respectively.

The confidence of normal traffic is basically close to 1, meaning that the detection of normal traffic mainly relies on the probability branch. In the CICIDS2018-III dataset, about 50% of the malicious traffic has a confidence level below 0.5. This suggests the confidence branch significantly contributes to the excellent performance of ECNet on this dataset. In the EnTra2023-III dataset, only a small part of the malicious traffic has low confidence. For malicious traffic with high confidence levels, ECNet accurately detects them by extracting representative features combined with the probability branch, achieving an F1 of 99.08%.

Combining these observations, we can conclude that both the probability and confidence branches of our ECNet are crucial. They complement each other, ensuring that ECNet maintains robust performance in various scenarios.

E. Complexity Analysis

To fully evaluate the trade-off between the model's performance and complexity, we compare the number of model parameters and the time required to analyze a session for ECNet and other methods, as shown in Table VII.

Rosetta has a significantly more parameters than other methods because its feature augmentation process requires the use of pre-trained models. ECNet employs both packet length sequences and raw packets, yet its parameter count is lower than that of FSNet, which uses packet length sequences, and GTID, which uses raw packets. Additionally, ECNet has a shorter runtime and superior detection performance compared to these methods, indicating that its model design is both concise and efficient. Compared to ACID and CVAE-EVT, ECNet has slightly higher parameter counts and runtime, mainly due to its processing of features from multiple perspectives. Nonetheless, ECNet's latency is acceptable and can achieve near real-time performance. ECNet leverages easily accessible features, including randomly sampled packets and session packet length and direction sequences, along with a simple model to reduce overhead.

F. Ablation Experiment

To quantify the individual contribution of each component in ECNet, we perform ablation experiments, and the results are shown in Table VIII and Table IX. For convenience, we denote using only content features and pattern features as 'w/ Content' and 'w/ Pattern'. Feature fusion is not required in these cases.

TABLE VII
NUMBER OF PARAMETERS AND TIME OVERHEAD

Method	Parameters	Times (ms)
FS-Net	2.25e+06	1.86e+02
CICAUTOML	-	1.28e-02
GTID	8.93e+05	3.53e+01
Kitsune	-	7.19e+00
Rosetta	1.14e+07	1.37e+02
ACID	1.27e+04	1.46e+00
CVAE-EVT	3.80e+04	1.75e+00
ECNet	1.25e+05	2.58e+00

TABLE VIII
ABLATION STUDY OF KEY COMPONENTS IN ECNET
ON THE CICIDS2018-I, II, III DATASET (%)

Dataset	Method	Accuracy	Precision	Recall	F1
I	w/ Content	99.87	99.37	99.99	99.68
	w/ Pattern	99.63	99.29	98.78	99.04
	w/o Fusion	99.89	99.46	99.99	99.73
	ECNet (default)	99.98	99.99	99.96	99.98
II	w/ Content	97.98	97.30	91.05	94.07
	w/ Pattern	92.44	92.63	61.89	74.20
	w/o Fusion	97.47	98.04	87.38	92.40
	ECNet (default)	98.97	95.83	93.38	94.59
III	w/ Content	95.99	94.05	62.32	74.96
	w/ Pattern	96.28	74.72	92.70	82.75
	w/o Fusion	96.87	95.09	71.10	81.37
	ECNet (default)	98.44	79.61	93.00	85.78

TABLE IX
ABLATION STUDY OF KEY COMPONENTS IN ECNET
ON THE ENTRA2023-I, II, III DATASET (%)

Dataset	Method	Accuracy	Precision	Recall	F1
I	w/ Content	99.89	99.43	99.99	99.71
	w/ Pattern	98.13	98.37	99.38	98.87
	w/o Fusion	99.94	99.93	99.99	99.96
	ECNet (default)	99.97	99.98	99.99	99.98
II	w/ Content	99.63	99.31	99.82	99.57
	w/ Pattern	94.92	91.59	96.88	94.16
	w/o Fusion	99.68	99.45	99.81	99.63
	ECNet (default)	99.82	99.87	99.70	99.79
III	w/ Content	98.79	98.86	98.18	98.52
	w/ Pattern	97.28	99.98	93.42	96.59
	w/o Fusion	99.35	99.75	98.68	99.21
	ECNet (default)	99.25	99.73	98.44	99.08

'w/o Fusion' represents not using gated feature fusion, but instead directly concatenating the two types of features.

Both content and pattern features play important roles. In most cases, content features are more crucial than pattern features. When detecting unknown malicious traffic, such as on the CICIDS2018-II dataset, using only pattern features results in a 6.53% decrease in accuracy and a 20.39% decrease in F1 for ECNet. Similar trends are observed on the EnTra2023-II dataset. Interestingly, on the CICIDS2018-III dataset, using only content features leads to a more significant performance drop, with a decrease of 10.82% in F1. This also shows that relying on a single type of feature is unstable and insufficient to cope with the variability of network traffic. Instead, the

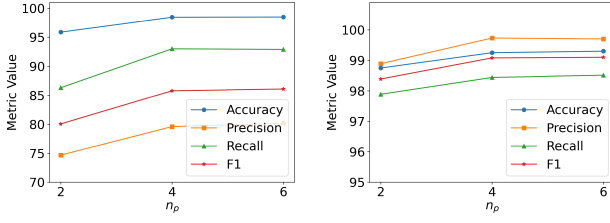


Fig. 9. Performance of ECNet with different number of randomly selected packets on the CICIDS2018-III dataset (left) and the EnTra2023-III dataset (right).

optimal performance is achieved by combining both content and pattern features.

Additionally, the use of gated feature fusion also plays a certain role in enhancing the performance of ECNet. On the CICIDS2018-III dataset, it results in an increase of 1.57% in accuracy and an improvement of 4.41% in F1.

G. Sensitivity Analysis

To investigate the impact of hyper-parameters on performance, we conduct a thorough sensitivity analysis of the key parameters in ECNet on the CICIDS2018-III dataset and EnTra2023-III dataset. The parameters analyzed include the number of randomly selected packets n_p , the dimension of encoded sequence vector v_{sq} m , the proportion of augmented samples α_c and α_p , and the confidence threshold for detecting malicious traffic δ_c .

1) *Number of Randomly Selected Packets*: ECNet randomly selects n_p packets from each session to extract content features. Figure 9 illustrates the performance of ECNet as n_p varies. The model performs poorly with only two packets, while the performance with six and four packets shows minimal differences. Typically, four packets are sufficient to include packets from both the session establishment and data transmission processes, so increasing to six packets does not yield substantial benefits. The inferior performance when using two packets can be attributed to two main factors. Firstly, two packets contain limited information. Secondly, it may be affected by the data augmentation process. During content augmentation, one of the n_p packet vectors is randomly set to a zero vector. When using only two packets, half of the content features are set to zero, potentially negatively impacting the model's training.

2) *Dimension of Encoded Sequence Vector*: ECNet encodes packet length and direction sequences into a vector v_{sq} with m columns. Figure 10 shows the impact of m on model performance. The results indicate that the choice of m has minimal impact on performance, suggesting that our encoding method combined with SessionCNN can effectively capture the pattern features of interaction processes. However, the choice of m affects the size of v_{sq} , which in turn influences the time and space overhead. In v_{sq} , each row stores the lengths of consecutive packets in the same direction (burst), while adjacent rows store the lengths of packets in different directions. If the length of a burst exceeds m , a blank row consisting of zeros needs to be filled in; if a burst is shorter than m , it is also padded with zeros to fill up to m . Figure 11 provides statistics of the burst lengths of the two datasets,

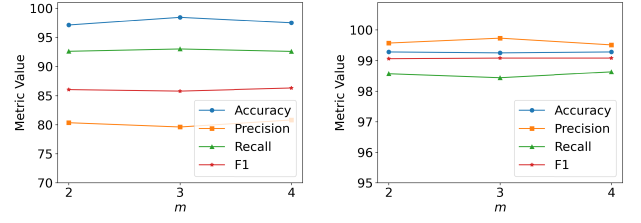


Fig. 10. Performance of ECNet with different dimension of encoded sequence vector on the CICIDS2018-III dataset (left) and the EnTra2023-III dataset (right).

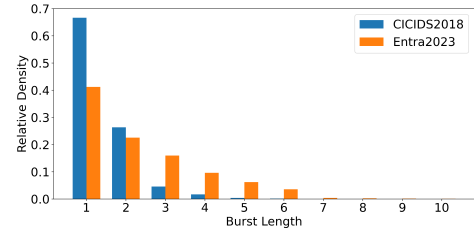


Fig. 11. Distribution of burst lengths in the CICIDS2018 dataset and the EnTra2023 dataset.

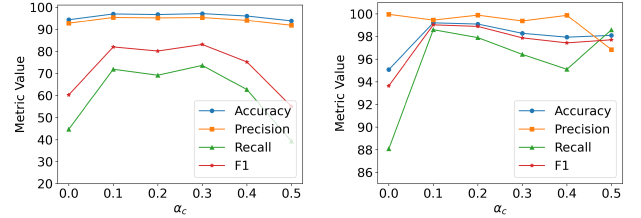


Fig. 12. Performance of ECNet with different proportion of sessions performed content augmentation on the CICIDS2018-III dataset (left) and the EnTra2023-III dataset (right).

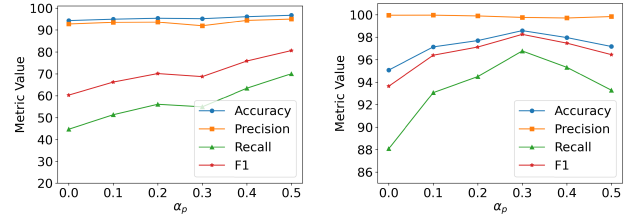


Fig. 13. Performance of ECNet with different proportion of sessions performed pattern feature augmentation on the CICIDS2018-III dataset (left) and the EnTra2023-III dataset (right).

showing that the burst length less than or equal to 3 can cover about 80% of the data, so setting m to 3 is enough.

3) *Proportion of Augmented Samples*: Data augmentation includes content augmentation on the data comprising a proportion of α_c and pattern augmentation on the data comprising a proportion of α_p during training. To more clearly analyze the effect of augmenting these two types of features, we avoid augmenting pattern features when adjusting the sample proportion for content feature augmentation, and vice versa. Figure 12 and Figure 13 respectively show the impact of the value of α_c and α_p on the performance of ECNet.

From Figure 12, we observe that ECNet performs best when 10% of the samples are performed content augmentation, and there is a significant improvement compared to the scenario without data augmentation ($\alpha_c = 0$). However, as α_c continues

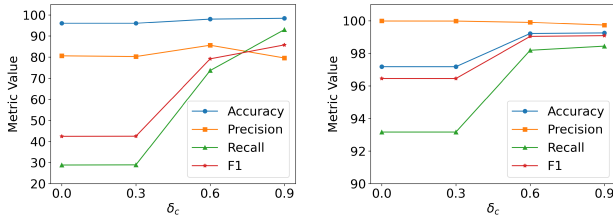


Fig. 14. Performance of ECNet with different confidence threshold on the CICIDS2018-III dataset (left) and the EnTra2023-III dataset (right).

to increase, the model's performance begins to decline. This is because content augmentation involves replacing all the values of a data packet with zeros, which has a considerable impact. A continual increase in α_c leads to an increase in difficult samples, resulting in unstable gradients during model training and hindering the learning of effective features.

The impact of pattern augmentation on performance is not as significant as that of content augmentation. From Figure 13, it can be observed that ECNet achieves satisfactory performance when 30% to 50% of the samples are performed pattern augmentation. During the training process, our ECNet performs both content and pattern augmentation on the training data to optimize the model's learning capability.

4) *Confidence Threshold*: Traffic with a confidence lower than δ_c are considered malicious. Figure 14 shows the variation in ECNet's performance as δ_c changes. When δ_c is 0, the confidence branch does not play a role. At this point, the model misclassifies many malicious traffic as normal, particularly resulting in poor performance on the CICIDS2018-III dataset. As δ_c increases, more and more unknown malicious traffic is correctly classified. When δ_c continues to increase from 0.6 to 0.9, there is a slight decrease in precision, as a small number of unfamiliar normal sessions are misclassified as malicious. However, this situation is rare and has minimal impact on overall performance.

VI. DISCUSSION

A. Challenges for Attackers in Evading Detection

Attackers may attempt to evade detection by modifying packets or altering the speed of traffic to mimic the characteristics of normal traffic. ECNet uses a deep learning approach. Attackers must understand normal network traffic over time and know the structure and settings of our model to evade detection. Furthermore, our method involves the random selection of packets, and the use of packet length and direction sequences. Attackers need to modify both aspects simultaneously while ensuring the effectiveness of their attack, which is very difficult. The random selection of packets means that the attacker needs to modify all packets. The combination of packet length and direction sequences requires attackers to control not only the packets they send but also those sent by the target host. This significantly heightens the challenge for attackers aiming to manipulate these features.

B. Usability in Changing Environment

In slightly changing network environments, ECNet shows good performance without being affected. In experiments,

we use normal data from different time periods for training and testing, ECNet exhibits high confidence in data, and the probability branch also accurately classifies the data, indicating that our method can resist slight changes well. Furthermore, in the future, we can combine ECNet with incremental learning. This allows ECNet to continuously learn new normal traffic patterns and continue to update and accumulate knowledge. This is different from attack data information leakage [30]. New attacks are unpredictable and cannot be obtained in advance, while normal traffic can be obtained continuously, so incremental learning is a feasible solution. In situations of drastic network changes, all methods face the challenge of retraining models by capturing a new dataset over a period and retraining the model, which is a common practice.

VII. CONCLUSION

In this paper, we propose a robust malicious traffic detection method with multi-view feature extraction and confidence mechanism, which can effectively and accurately detect malicious traffic in the real world. ECNet extracts multi-view features from sessions, including content features and pattern features. A gated-based feature fusion method is also used to combine these features. ECNet introduces a confidence mechanism. For each session model, it not only outputs the probability that it belongs to each category but also outputs a confidence value, indicating the reliability of this result. Combining these two outputs, ECNet can accurately detect known and unknown attacks. We conduct comprehensive experiments on six reorganized datasets, and the results demonstrate that our ECNet outperforms other state-of-the-art methods in detecting both known and unknown malicious traffic.

In the future, we will explore integrating ECNet with incremental learning to further enhance its robustness. We will also investigate the adaptive configuration of the confidence threshold, making it more automated.

REFERENCES

- [1] D. Ucci, L. Aniello, and R. Baldoni, "Survey of machine learning techniques for malware analysis," *Comput. Secur.*, vol. 81, pp. 123–147, Mar. 2019.
- [2] J. Gu and S. Lu, "An effective intrusion detection approach using SVM with naïve Bayes feature embedding," *Comput. Secur.*, vol. 103, Apr. 2021, Art. no. 102158.
- [3] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Halifax, NS, Canada, Aug. 2017, pp. 1723–1732.
- [4] X. Han et al., "STIDM: A spatial and temporal aware intrusion detection model," in *Proc. IEEE 19th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2020, pp. 370–377.
- [5] L. Yu et al., "PBCNN: Packet bytes-based convolutional neural network for network intrusion detection," *Comput. Netw.*, vol. 194, Jul. 2021, Art. no. 108117.
- [6] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Sep. 2019, pp. 1171–1179.
- [7] X. Han, S. Cui, S. Liu, C. Zhang, B. Jiang, and Z. Lu, "Network intrusion detection based on n-gram frequency and time-aware transformer," *Comput. Secur.*, vol. 128, May 2023, Art. no. 103171.
- [8] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2066–2075.

- [9] Z. Fu et al., "Encrypted malware traffic detection via graph-based network analysis," in *Proc. 25th Int. Symp. Res. Attacks, Intrusions Def.*, 2022, pp. 495–509.
- [10] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
- [11] M. Catillo, A. Pecchia, and U. Villano, "CPS-GUARD: Intrusion detection for cyber-physical systems and IoT devices using outlier-aware deep autoencoders," *Comput. Secur.*, vol. 129, Jun. 2023, Art. no. 103210.
- [12] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.
- [13] P. Zhang et al., "Real-time malicious traffic detection with online isolation forest over SD-WAN," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2076–2090, 2023.
- [14] A. F. Diallo and P. Patras, "Adaptive clustering-based malicious traffic classification at the network edge," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2021, pp. 1–10.
- [15] R. Xie et al., "Rosetta: Enabling robust TLS encrypted traffic classification in diverse network environments with TCP-aware traffic augmentation," in *Proc. ACM Turing Award Celebration Conf. China*, Jul. 2023, pp. 625–642. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/xie>
- [16] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3431–3446.
- [17] J. Yang, X. Chen, S. Chen, X. Jiang, and X. Tan, "Conditional variational auto-encoder and extreme value theory aided two-stage learning approach for intelligent fine-grained known/unknown intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3538–3553, 2021.
- [18] Z. Qiu, D. Zhou, Y. Zhai, B. Liu, L. He, and J. Cao, "VAEMax: Open-set intrusion detection based on OpenMax and variational autoencoder," 2024, *arXiv:2403.04193*.
- [19] X. Hu, W. Gao, G. Cheng, R. Li, Y. Zhou, and H. Wu, "Toward early and accurate network intrusion detection using graph embedding," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5817–5831, 2023.
- [20] D. Suci, S. McLaughlin, L. Simon, and R. Sion, "Horizontal privilege escalation in trusted applications," in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)*, 2020, pp. 825–840.
- [21] S. Siby, M. Juarez, C. Diaz, N. Vallina-Rodriguez, and C. Troncoso, "Encrypted DNS-> privacy? A traffic analysis perspective," 2019, *arXiv:1906.09682*.
- [22] S. Cui, C. Dong, M. Shen, Y. Liu, B. Jiang, and Z. Lu, "CBSeq: A channel-level behavior sequence for encrypted malware traffic detection," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5011–5025, 2023.
- [23] K. Al-Naami et al., "Adaptive encrypted traffic fingerprinting with bi-directional dependence," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, Dec. 2016, pp. 177–188.
- [24] T. DeVries and G. W. Taylor, "Learning confidence for out-of-distribution detection in neural networks," 2018, *arXiv:1802.04865*.
- [25] Y. Lu, J. Zeng, J. Zhang, S. Wu, and M. Li, "Learning confidence for transformer-based neural machine translation," 2022, *arXiv:2203.11413*.
- [26] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2018, pp. 108–116.
- [27] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, "New directions in automated traffic analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3366–3383.
- [28] (2023). *Automl for Image, Text, Time Series, and Tabular Data*. Accessed: Dec. 7, 2023. [Online]. Available: <https://auto.gluon.ai/stable/index.html>
- [29] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Porto, Portugal, Feb. 2017, pp. 253–262.
- [30] E. Quiring et al., "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Secur. Symp. (USENIX Secur.)*, 2022, pp. 3971–3988.



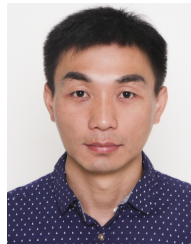
Xueying Han received the B.S. degree from Beijing University of Posts and Telecommunications, Beijing, China, in 2020. She is currently pursuing the Ph.D. degree with the Institute of Information Engineering, University of Chinese Academy of Sciences, Beijing. Her research interests include deep learning and intrusion detection.



Song Liu received the M.S. degree from the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China, in 2018. He is currently an Engineer with the Institute of Information Engineering, Chinese Academy of Sciences. His current research interests include network attack detection and defense, and big data processing.



Junrong Liu received the master's degree from Beijing University of Posts and Telecommunications in 2010. She is currently a Senior Engineer with the Institute of Information Engineering, Chinese Academy of Sciences. Her research interests include network situational awareness and network security visualization.



Bo Jiang received the Ph.D. degree from the Chinese Academy of Sciences in 2016. He is currently an Assistant Professor with the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include network situational awareness, knowledge graphs, and data mining.



focus on cyber situation awareness, intrusion detection and prevention, and mobile terminal security.

Zhigang Lu received the Ph.D. degree in computer applications from the University of Chinese Academy of Sciences. He is currently a Professor with the Department of Cyber Security, University of Chinese Academy of Sciences. He has published more than 30 papers in top network and security journals and conferences, including IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, *Computer Networks*, *Computers & Security*, and *TrustCom*. His research interests include network and system security, with a specific



Baoxu Liu received the Ph.D. degree from the Graduate School of the Chinese Academy of Sciences, Beijing, China, in 2002. He is currently a Professor and a Ph.D. Supervisor with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, and the School of Cyber Security, University of Chinese Academy of Sciences, Beijing. His current research interests include network attack detection and defense, and network security situational awareness.